

PROC SQL

PERFORMING QUERIES USING PROC SQL

Background – SQL history

- Background – SQL history
- Structured Query Language.
- Developed by IBM in the 1970s for querying,modifying and updating relational databases
- Used with almost all RDBMS (Relational Data Base Management System) e.g. Oracle, DB2,Access etc.
- PROC SQL is SAS implementation of Structured Query Language (SQL), which is a standardized language that is widely used to retrieve and update data in tables and in views that are based on those tables.

RangTech
Enhancing the way IT works

WHY USE SQL?

Sometimes you need quick answers to questions about your data. You might want to query (retrieve data from) a single SAS data set or a combination of data sets to

- examine relationships between data values
- view a subset of your data
- compute values quickly.
- The SQL procedure (PROC SQL) provides an easy, flexible way to query and combine your data.

Rang Tech
Enhancing the way IT works

COMPARING DATA STEP AND SQL TERMS:

The following chart shows terms used in data processing, SAS, and SQL that are synonymous. The SQL terms are used in this lesson.

Data Processing	SAS	SQL
file	SAS data set	table
record	observation	row
field	variable	column

How PROC SQL Is Unique

- Unlike other PROC statements, many statements in PROC SQL are composed of **clauses**
- Many statements in PROC SQL, such as the SELECT statement, are composed of clauses.
- The PROC SQL step does not require a RUN statement.
- PROC SQL continues to run after you submit a step. To end the procedure, you must submit another PROC step, a DATA step, or a QUIT statement.

Ranglech
Enhancing the way IT works

- **PROC SQL;**
  SELECT *column-1*<, . . . *column-n*>
  FROM *table-1* | *view-1*<, ... *table-n* | *view-n*>
  <**WHERE** *expression*>
  <**GROUP BY** *column-1*<, .. *column-n*>>
  <**ORDER BY** *column-1*<, ... *column-n*>>**;** quit;

RanaTech
Embracing the way IT works

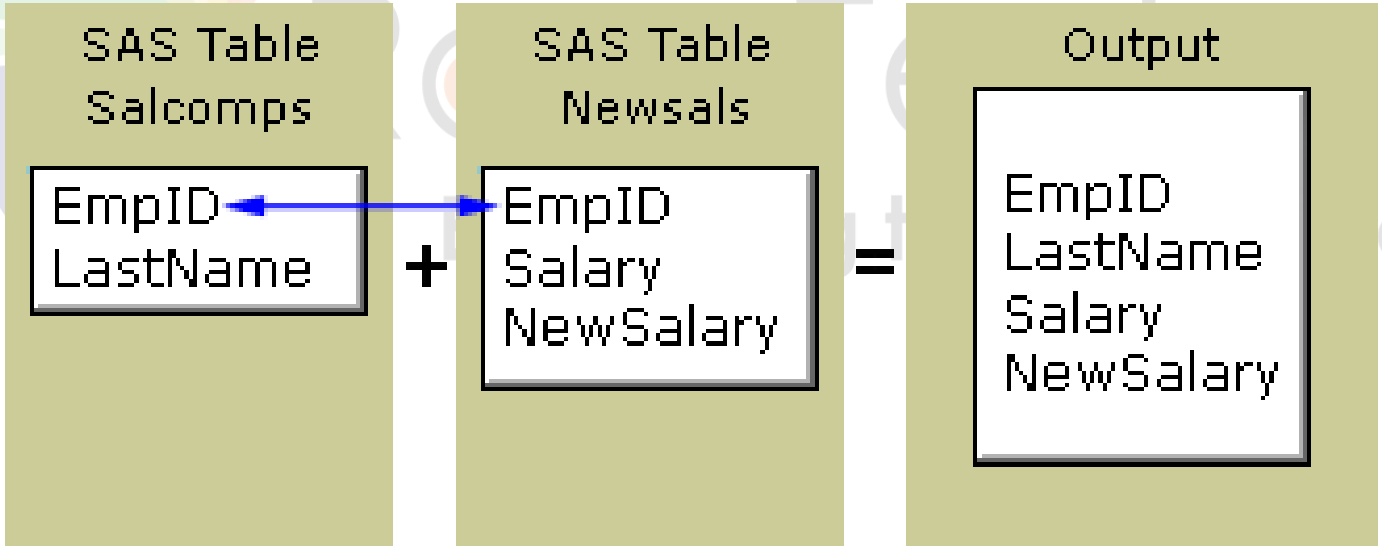
GENERAL FORM OF QUERY

- **PROC SQL** invokes the SQL procedure
- **SELECT** specifies the column(s) to be selected
- **FROM** specifies the table(s) to be queried
- **WHERE** subsets the data based on a condition
- **GROUP BY** classifies the data into groups based on the specified column(s)
- **ORDER BY** sorts the rows that the query returns by the value(s) of the specified column(s).
- A query can also include a HAVING clause, which is introduced at the end of this lesson

RonTech
Enhancing the way IT works

Querying Multiple Tables

You can use a PROC SQL step to query data that is stored in two or more tables. In SQL terminology, this is called **joining** tables



HOW TO USE PROC SQL

- **Selecting Columns**
  To specify which column(s) to display in a query, you write a SELECT clause as the first clause in the SELECT statement. In the SELECT clause, you can specify existing columns and create new columns that contain either text or a calculation.

RanTech
Enhancing the way it works

HOW TO USE PROC SQL CONT.

- **Specifying the Table**
  You specify the table to be queried in the FROM clause.
  Specifying Subsetting Criteria
  To subset data based on a condition, write a WHERE clause that contains an expression.
  Ordering Rows
  The order of rows in the output of a PROC SQL query cannot be guaranteed, unless you specify a sort order. To sort rows by the values of specific columns, use the ORDER BY clause.

RapidLearn
Enhancing the way IT works

Sample Programs Querying a Table

- proc sql;
- select empid,jobcode,salary, salary*.06 as bonus
- from sasuser.payrollmaster
- where salary<32000
- order by jobcode; quit;

RangTech
Enhancing the way IT works

Summarizing Groups of Data

- **Summarizing Groups of Data**
  You can use a GROUP BY clause in your PROC SQL step to summarize data in groups..
  EXAMPLE;
- proc sql;
  select membertype, sum(milestraveled) as TotalMiles
  from sasuser.frequentflyers
  group by membertype;
- quit;

RanaTech
Enhancing the way IT works

Querying Multiple Tables

- **Querying Multiple Tables**
  You can use a PROC SQL step to query data that is stored in two or more tables. In SQL terminology, this is called **joining** tables

Ranglech
Enhancing the way IT works

EXAMPLE OF JOINING TABLES

- proc sql;
- **select** salcomps.empid,lastname, newsals.salary,newsalary
- from sasuser.salcomps,sasuser.newsals **where** salcomps.empid=newsals.empid order by lastname ;
- QUIT;

RonaTech
Enhancing the way IT works

Points to Remember

- You do not use a RUN statement with the SQL procedure.
- Do not end a clause with a semicolon unless it is the last clause in the statement.
- When you join multiple tables, be sure to specify columns that have matching data values in the WHERE clause in order to avoid unwanted combinations.
- To end the SQL procedure, you can submit another PROC step, a DATA step, or a QUIT statement.

RangTech
Enhancing the way it works

PROC SQL

RangTech
Enhancing the way IT works

Performing Advanced Queries Using PROC SQL

Displaying All Columns

- To display all columns in the order in which they are stored in the table, use an asterisk (*) in the SELECT clause. To write the expanded list of columns to the SAS log, use the FEEDBACK option in the PROC SQL statement.
- Example:

```
proc sql feedback;
select *
from sasuser.staffchanges;
quit;
```

RangTech
Enhancing the way IT works

Limiting the Number of Rows Displayed

- To limit the number of rows that PROC SQL displays as output, use the OUTOBS=*n* option in the PROC SQL statement.
- **Eliminating Duplicate Rows from Output**
  To eliminate duplicate rows from your query results, use the keyword DISTINCT in the SELECT clause.
- Example:

```
proc sql;
select distinct flightnumber, destination
from sasuser.internationalflights
order by 1; quit;
```

RanTech
Enhancing the way IT works

Sub setting Rows by Using Conditional Operators

- In a PROC SQL query, use the WHERE clause with any valid SAS expression to subset data. The SAS expression can contain one or more operators, including the following conditional operators:
- the BETWEEN-AND operator selects within an inclusive range of values
- the CONTAINS or ? operator selects a character string
- the IN operator selects from a list of fixed values
- the IS MISSING or IS NULL operator selects missing values
- the LIKE operator selects a pattern
- the sounds-like (=*) operator selects a spelling variation

RangTech
Enhancing the way IT works

Sub setting Rows by Using Calculated Values

You can also use a calculated column in the WHERE clause to subset rows. However, because of the way in which SQL queries are processed, you cannot just specify the column alias in the WHERE clause.

- **Using the Keyword CALCULATED**
- When you use a column alias in the WHERE clause to refer to a calculated value, you must use the keyword **CALCULATED** along with the alias. The CALCULATED keyword informs PROC SQL that the value is calculated within the query.
- proc sql outobs=10;

  select flightnumber, date, destination, boarded + transferred + nonrevenue as Total

  from sasuser.marchflights

  where **calculated** total < 100;

Enhancing Query Output

You can enhance PROC SQL query output by using SAS enhancements such as column formats and labels, titles and footnotes, and character constants.

Your first task is to specify column labels for the first two columns. Below, the **LABEL=** option has been added after both EmpID and JobCode, and the text of each label is enclosed in quotation marks.

```
proc sql outobs=15;
 select empid label='Employee ID', jobcode
label='Job Code', salary, salary * .10 as Bonus
from sasuser.payrollmaster where salary>75000
order by salary desc;
```

RamTech Enhancing the way IT works

Enhancing Query Output cont.

- Next, you will add a format for the Bonus column. Because the Bonus values are dollar amounts, you will use the format **Dollar12.2**. The **FORMAT=** option has been added to the SELECT clause, below, immediately following the column alias Bonus:

- proc sql outobs=15;

  select empid label='Employee ID', jobcode label='Job Code', salary, salary * .10 as Bonus **format=dollar12.2** from sasuser.payrollmaster where salary>75000 order by salary desc;

RangTech
Enhancing the way IT works

TITLES AND FOOTNOTES

- When you specify titles and footnotes with a PROC SQL query, you must place the TITLE and FOOTNOTE statements in either of the following locations:
- before the PROC SQL statement
- between the PROC SQL statement and the SELECT statement.

```
proc sql outobs=15;
title 'Current Bonus Information'; title2 'Employees with Salaries > $75,000';
select empid label='Employee ID', jobcode label='Job Code', salary, salary * .10 as Bonus format=dollar12.2 from sasuser.payrollmaster where salary>75000 order by salary desc;
```

RangTech
Enhancing the way IT works

Adding a Character Constant to Output

- To specify a new column that contains a character constant, you include the text string in quotation marks in the SELECT clause list.

RangTech
Enhancing the way IT works

PROC SQL



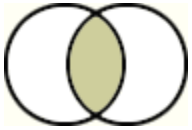
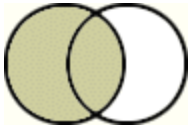
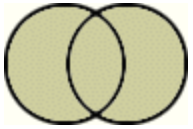
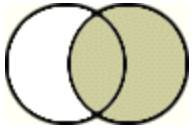
Combining Tables Horizontally Using PROC SQL

RangTech
Enhancing the way IT works

UNDERSTANDING JOINS

- Joins combine tables horizontally (side by side) by combining rows. The tables being joined are not required to have the same number of rows or columns.
  You can use a join to combine views as well as tables.

Rangtech
Enhancing the way IT works

Type of Join	Output
Inner join 	Only the rows that **match** across all table(s)
Outer join   	Rows that **match** across tables (as in the inner join) plus **nonmatching** rows from one or more tables

Generating a Cartesian Product

- When you specify multiple tables in the FROM clause but do not include a WHERE statement to subset data, PROC SQL returns the Cartesian product of the tables.
- In a Cartesian product, each row in the first table is combined with every row in the second table.
- In all types of joins, PROC SQL generates a Cartesian product first, and then eliminates rows that do not meet any subsetting criteria that you have specified.

RangTech
Enhancing the way IT works

Using Inner Joins

- An inner join combines and displays the rows from the first table that match rows from the second table, based on the matching criteria (also known as join conditions) that are specified in the WHERE clause.
- When the tables that are being joined contain a column with a common name, you might want to eliminate the duplicate column from results or specify a column alias to rename one of the duplicate columns.
- To refer to tables in an inner join, or in any PROC SQL step, you can specify a temporary name called a table alias.

RangeTech
Enhancing the way IT works

Combining Tables by Using an Inner Join

- proc sql outobs=15;
- title 'New York Employees';
- select substr(firstname,1,1) || '. ' || lastname as Name, jobcode, int((today() - dateofbirth)/365.25) as Age
- from sasuser.payrollmaster as p, sasuser.staffmaster as s
- where p.empid = s.empid and state='NY' order by 2, 3; quit;

RapidTech
Enhancing the way IT works

Using Outer Joins

- An outer join combines and displays all rows that match across tables, based on the specified matching criteria (also known as join conditions), plus some or all of the rows that do not match. There are three types of outer joins:
- A left outer join retrieves all rows that match across tables, based on the specified matching criteria (join conditions), plus nonmatching rows from the left table (the first table specified in the FROM clause).
  A right outer join retrieves all rows that match across tables, based on the specified matching criteria (join conditions), plus nonmatching rows from the right table (the second table specified in the FROM clause).
  A full outer join retrieves both matching rows and nonmatching rows from both tables.

Ranglen
Enhancing the way IT works

Comparing SQL Joins and DATA Step Match-Merges

- DATA step match-merges and PROC SQL joins can produce the same results, although there are important differences between these two techniques.
- When **all** the values of the BY variable (column) match and there are no duplicate BY variables, you can use a PROC SQL inner join.

- When only **some** of the values of the BY variable match, you can use a PROC SQL full outer join. To overlay common columns, you must use the COALESCE function with the PROC SQL join.
- There are several advantages to using PROC SQL joins.

RangTech
Enhancing the way IT works

Comparing SQL Joins and DATA Step Match-Merges (continued)

Understanding Advantages of PROC SQL Joins

DATA step match-merges and PROC SQL joins both have advantages and disadvantages. Here are some of the main advantages of PROC SQL joins.

Join performance can be substantially improved when the tables are indexed on the column(s) on which the tables are being joined.

Advantage	Example
PROC SQL joins do **not** require **sorted or indexed** tables.	proc sql; select table1.x, a, b from table1 full join table2 on table1.x = table2.x; where **table-1** is **sorted** by column X and **table-2** is **not**
PROC SQL joins do **not** require that the columns in join expressions have the **same name**.	proc sql; select table1.x, lastname, status from table1, table2 where **table1.id = table2.custnum**;
PROC SQL joins can use comparison operators **other than the equal sign (=).**	proc sql; select a.itemnumber, cost, price from table1 as a, table2 as b where a.itemnumber = b.itemnumber and **a.cost>b.price**;

Comparing SQL Joins and DATA Step Match-Merges (continued)

- **When Only Some of the Values Match: Using the COALESCE Function**
- When you add the COALESCE function to the SELECT clause of the PROC SQL outer join, the PROC SQL outer join can produce the same result as a DATA step match-merge.
  The COALESCE function overlays the specified columns by
- checking the value of each column in the order in which the columns are listed
- returning the first value that is a SAS nonmissing value.
- If all returned values are missing, COALESCE returns a missing value.

- Below is the SELECT clause from the PROC SQL full outer join shown on the previous page, with the COALESCE function added:
  select **coalesce(three.x, four.x)** as X, a, b
- This SELECT clause overlays the X columns from tables **Three** and **Four**. The column alias X is assigned to the coalesced column.
- When the COALESCE function is added to the preceding PROC SQL full outer join, the DATA step match-merge (with PROC PRINT step) and the PROC SQL full outer join will combine rows in the **same** way

RangTech
Changing the way IT works

Table Merged

X	A
1	a
2	b
4	d

Three

X	B
2	x
3	y
5	v

X	A	B
1	a	
2	b	x
3		y
4	d	
5		v

Ranatech
Enhancing the way IT works

The COALESCE function

- **DATA Step Match-Merge**
- data merged; merge three four; by x; run; proc print data=merged noobs; title 'Table Merged'; run;

- **PROC SQL Full Outer Join**
- proc sql; title 'Table Merged'; select **coalesce(three.x, four.x) as X**, a, b from three full join four on three.x = four.x;

Rang-tech
Enhancing the way IT works

Combining Tables by Using a Left Outer Join

- proc sql outobs=20; title 'All March Flights'; select m.date, m.flightnumber label='Flight Number', m.destination label='Left', f.destination label='Right', delay label='Delay in Minutes' from sasuser.marchflights as m left join sasuser.flightdelays as f on m.date=f.date and m.flightnumber= f.flightnumber order by delay; quit;

Rangtech
Enhancing the way IT works

Overlaying Common Columns in a Full Outer Join

- proc sql; select coalesce(p.empid, s.empid) as ID, firstname, lastname, gender from sasuser.payrollmaster as p full join sasuser.staffmaster as s on p.empid = s.empid order by id; quit;

RangTech
Enhancing the way IT works

Joining Tables by Using a Subquery and an In-line View

```
proc sql; select empid from sasuser.supervisors
  as m, (select substr(jobcode,1,2) as
  JobCategory, state from sasuser.staffmaster as
  s, sasuser.payrollmaster as p where
  s.empid=p.empid and s.empid in (select empid
  from sasuser.flightschedule where
  date='04mar2000'd and destination='CPH')) as c
  where m.jobcategory=c.jobcategory and
  m.state=c.state; quit;
```

RanTech
Enhancing the way IT works

Using In-Line Views

An in-line view is a subquery that appears in a FROM clause. An in-line view selects data from one or more tables to produce a temporary (or virtual) table that the outer query uses to select data for output. You can reference an in-line view with other views or tables, reference multiple tables in an in-line view, and assign an alias to an in-line view.

Points to Remember

- In most cases, generating all possible combinations of rows from multiple tables does not yield useful results, so a Cartesian product is rarely the query outcome that you want.

- You can combine a maximum of 32 tables in a single inner join. (If the join involves views, it is the number of tables that underlie the views, not the number of views, that counts towards the limit of 32.) An outer join can be performed on only two tables or views at a time.

Ranglech
Enhancing the way IT works

Understanding Set Operations

A set operation combines tables or views vertically (one on top of the other) by combining the results of two queries. A set operation is a SELECT statement that contains

EX-1 - proc sql; select * from a **set-operator** select * from b;

EX-2 - proc sql; **select** * from sasuser.stress98 **union select** * from sasuser.stress99

two groups of query clauses (each group beginning with a SELECT clause)

one of the set operators EXCEPT, INTERSECT, UNION, and OUTER UNION

optionally, one or both of the keywords ALL and CORR (CORRESPONDING) as modifiers.

EX-1 - proc sql; select * from table1 **except** select * from table2;

EX-2 - proc sql; select * from table1 **intersect** select * from table2;

EX-3 - proc sql; select * from table1 **union** select * from table2;

Ex – 4 - proc sql; select * from table1 **outer union** select * from table2;

EX -5 - proc sql; select * from table1 **set-operator <all> <corr>** select * from table2;

A single SELECT statement can contain multiple set operations.

EX - proc sql; **select** * from table1 **set-operator select** * from table2 **set-operator select** * from table3;

When processing a set operation that displays only unique rows (a set operation that contains the set operator EXCEPT, INTERSECT, or UNION), PROC SQL makes two passes through the data, by default. For set operations that display both unique and duplicate rows, only one pass through the data is required.

EX - proc sql; select * from one **except** select * from two;

For the set operators EXCEPT, INTERSECT, and UNION, columns are overlaid based on the relative position of the columns in the SELECT clause rather than by column name. In order to be overlaid, columns in the same relative position in the two SELECT clauses must have the same data type.

One or both keywords can be used to modify the default action of a set operator.

EX - proc sql; select * from one **except** select * from two;

Understanding Set Operations

Set operation is a SELECT statement that contains

- two groups of query clauses (each group beginning with a SELECT clause)
- a set operator
- optionally, one or both of the keywords ALL and CORR (CORRESPONDING).

RanaTech
Enhancing the way IT works

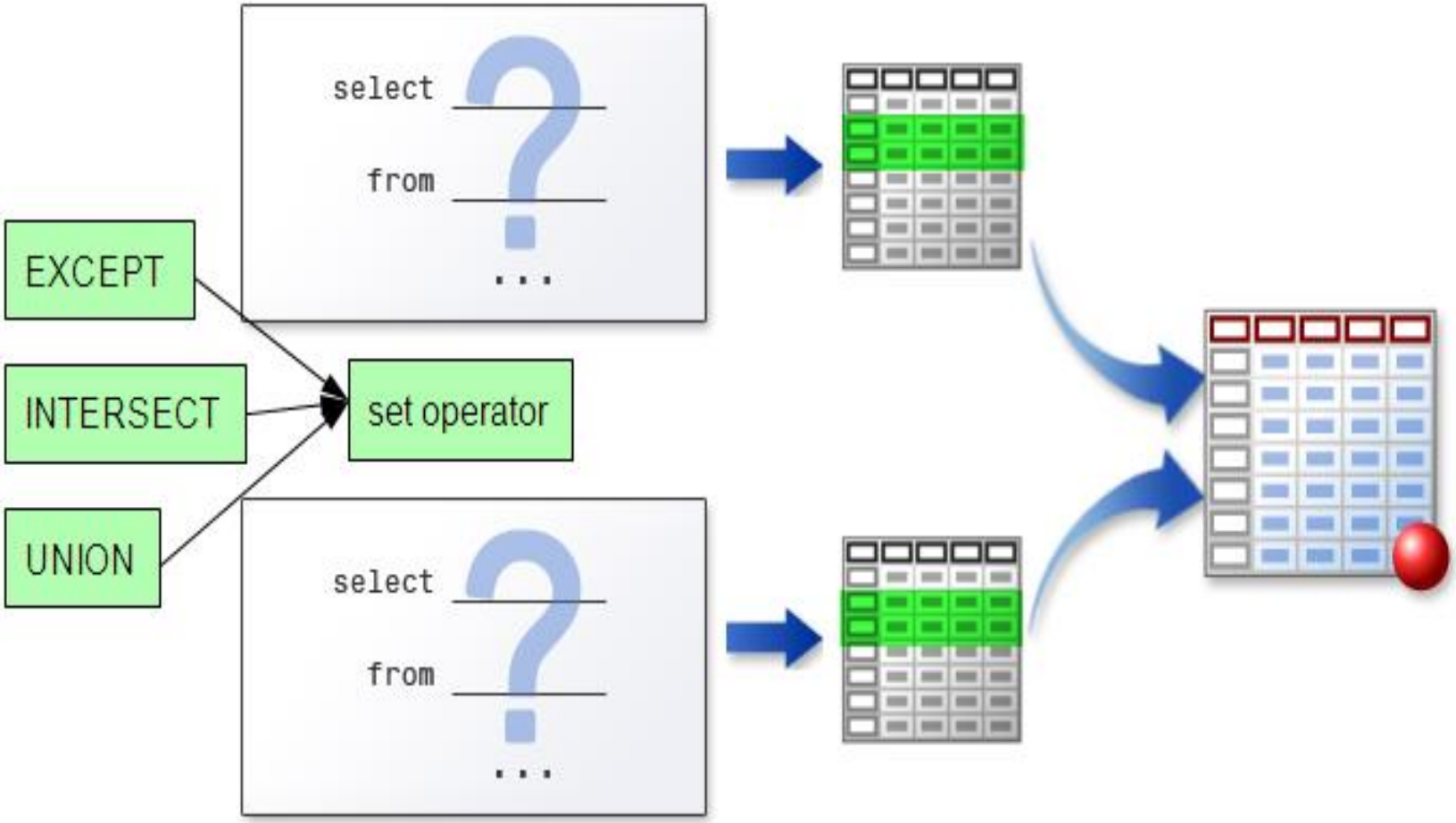
Example

- The set operation uses the set operator UNION to combine the result of a query on the table **Sasuser.Stress98** with the result of a query on the table **Sasuser.Stress99**.

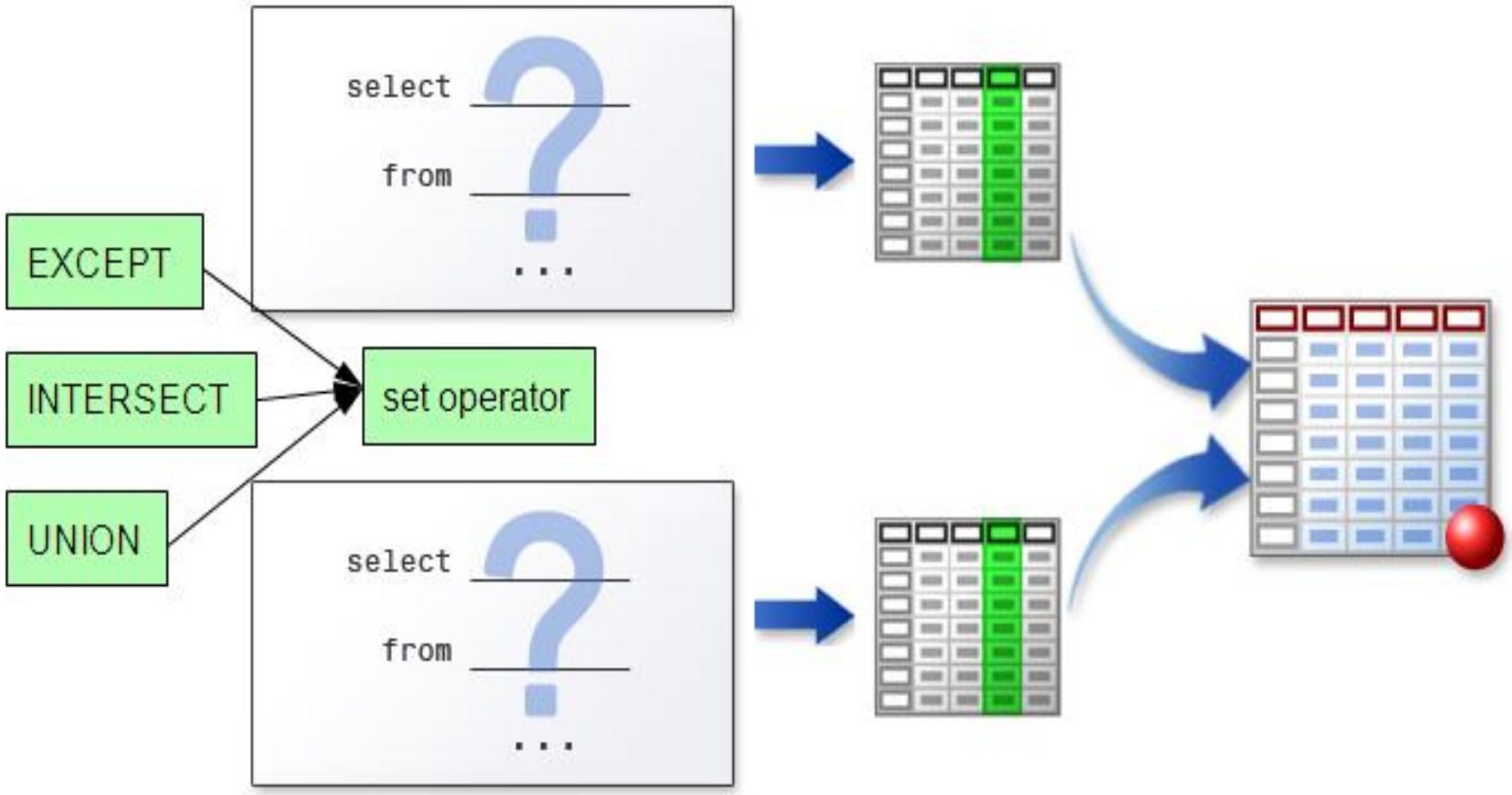
```
proc sql;
select * from sasuser.stress98 union
select * from sasuser.stress99;
```

RangTech
Enhancing the way IT works

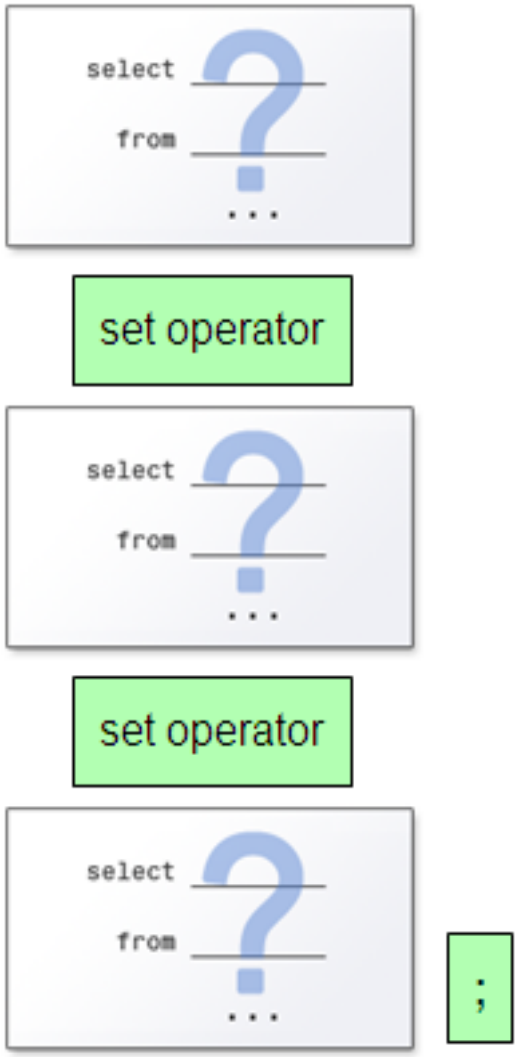
Evaluating Queries with Set Operators

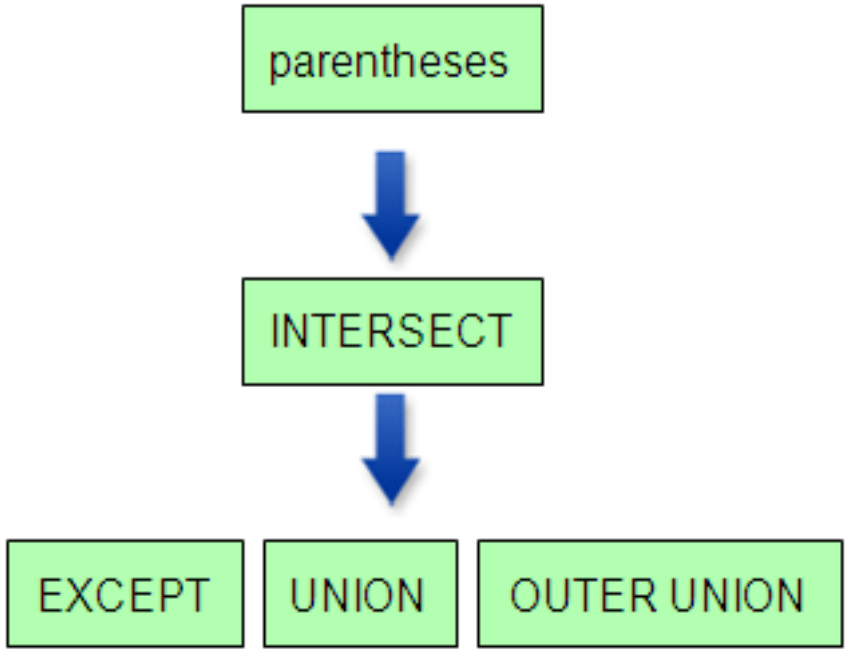


Evaluating Queries with Set Operators



Multiple Set Operations





Using the EXCEPT Set Operator

- The set operator EXCEPT selects unique rows from the first table (the table specified in the first query) that are not found in the second table (the table specified in the second query) and overlays columns. This set operation can be modified by using either or both of the keywords ALL and CORR.

EX – 1 proc sql; select * from one except select * from two;

EX – 2 proc sql; select * from one except **all** select * from two;

EX – 3 proc sql; select * from one except **corr** select * from two;

RanTech
Enhancing the way IT works

How the EXCEPT Operator Works

Intermediate Source Table **One**

X	A
1	a
1	a
1	b
2	c
3	v
4	e
6	g

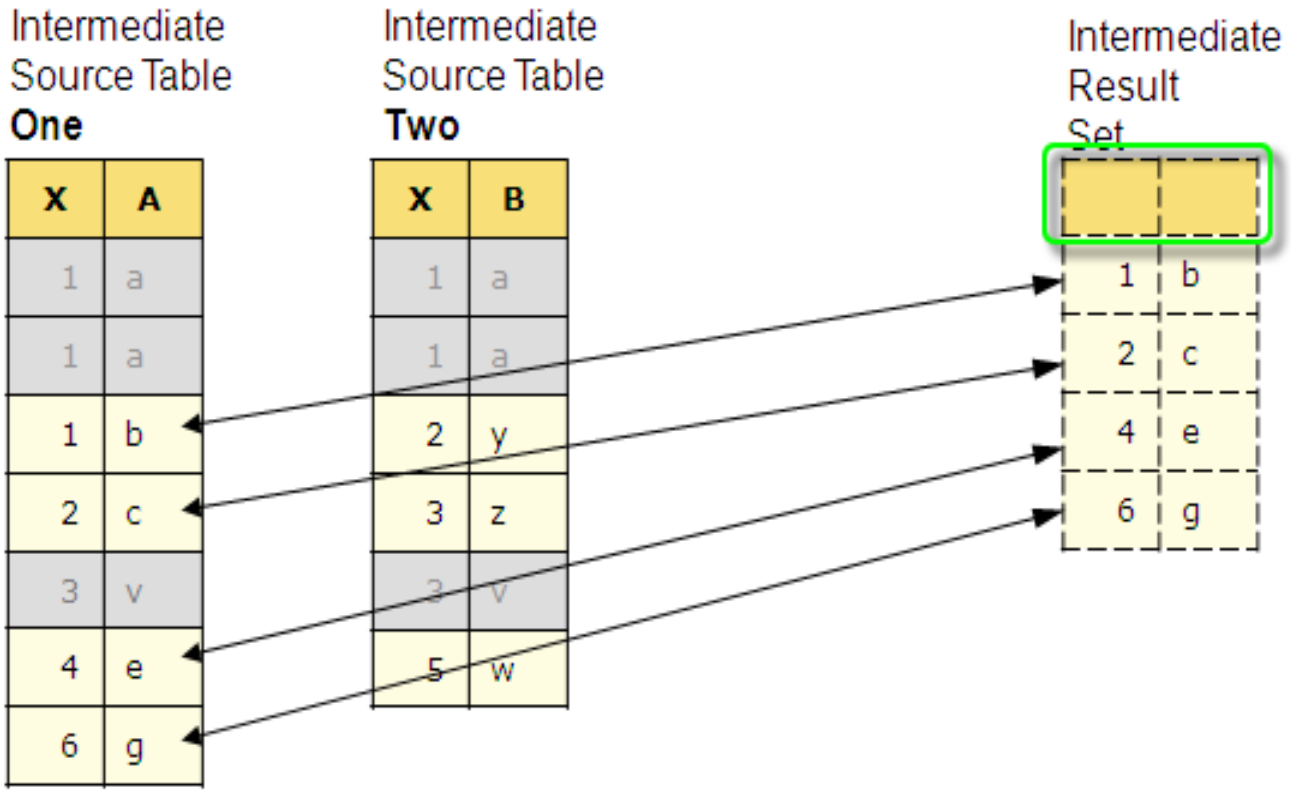
Intermediate Source Table **Two**

X	B
1	a
1	a
2	y
3	z
3	v
5	w

```
proc sql;
   select *
      from one
   except
   select *
      from two;
quit;
```

Pass	1	2

How the EXCEPT Operator Works



Pass	1	2	3

Using the Keyword ALL with the EXCEPT Operator

Intermediate Source Table **One**

X	A
1	a
1	a
1	b
2	c
3	v
4	e
6	g

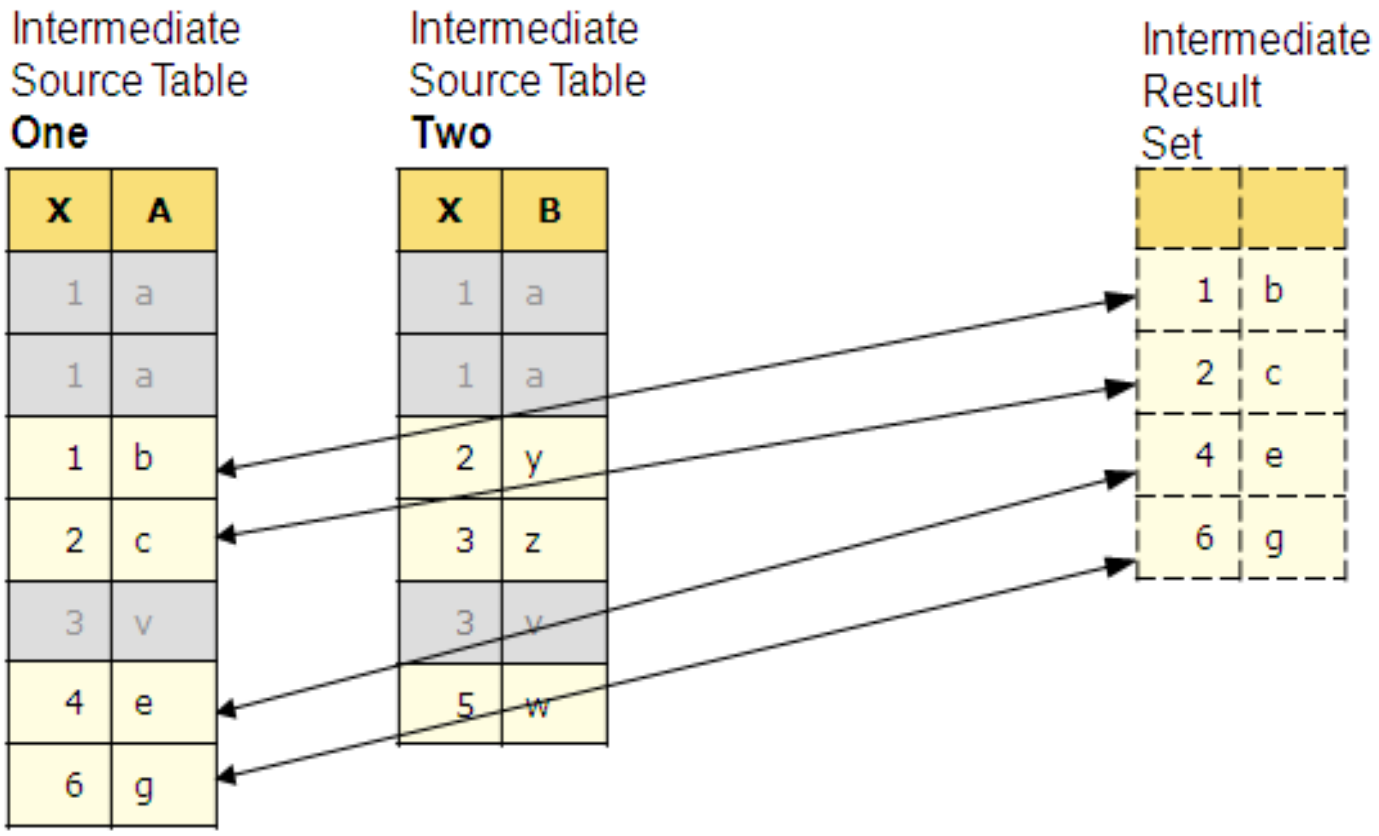
Intermediate Source Table **Two**

X	B
1	a
1	a
2	y
3	z
3	v
5	w

```
proc sql;
    select *
       from one
    except all
    select *
       from two;
quit;
```

Pass	1	2

Using the Keyword ALL with the EXCEPT Operator



Using the Keyword CORR with the EXCEPT Operator

Intermediate Source Table **One**

X	A
1	a
1	a
1	b
2	c
3	v
4	e
6	g

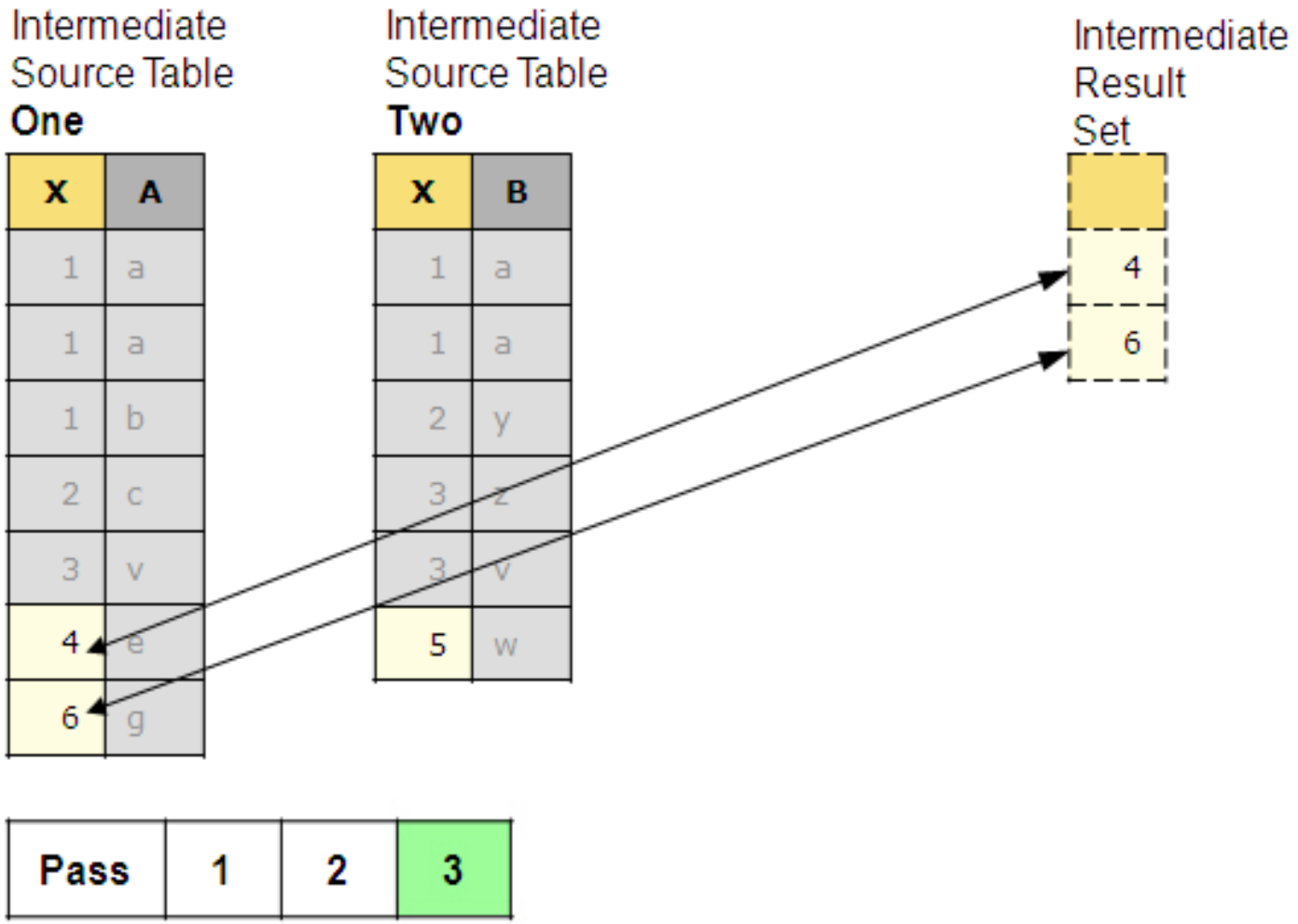
Intermediate Source Table **Two**

X	B
1	a
1	a
2	y
3	z
3	v
5	w

```
proc sql;
    select *
       from one
    except corr
    select *
       from two;
quit;
```

Pass	1	2

Using the Keyword CORR with the EXCEPT Operator



Using the INTERSECT Set Operator

- The set operator INTERSECT selects unique rows that are common to both tables and overlays columns. This set operation can be modified by using either or both of the keywords ALL and CORR.
- EX -1 proc sql; select * from one **intersect** select * from two;
- EX -2 proc sql; select * from one **intersect ALL** select * from two;
- EX – 3 proc sql; select * from one **intersect CORR** select * from two;

Using the UNION Set Operator

- The set operator UNION selects unique rows from both tables together and overlays columns. This set operation can be modified by using either or both of the keywords ALL and CORR
- EX -1 proc sql; select * from one **union** select * from two;
- EX – 2 proc sql; select * from one **union ALL** select * from two;
- EX – 3 proc sql; select * from one **union CORR** select * from two;

RangTech
Enhancing the way IT works

Using the OUTER UNION Set Operator

- The set operator OUTER UNION concatenates the results of two queries by selecting all rows (both unique and nonunique) from both tables and **not** overlaying columns. This set operation can be modified by using the keyword CORR.
- EX -1 proc sql; select * from one **outer union** select * from two;
- EX – 2 proc sql; select * from one **outer union corr** select * from two;

RonTech
Enhancing the way IT works

Comparing Outer Unions and Other SAS Techniques

- A PROC SQL set operation that uses the OUTER UNION set operator is not the only way to concatenate tables in SAS. Other SAS techniques can be used, such as a program that consists of a DATA step, a SET statement, and a PROC PRINT step.
- **Program 1: PROC SQL OUTER UNION Set Operation with CORR**
- proc sql; create table three as select * from one outer union corr select * from two; quit;

Ranatech
Enhancing the way IT works

PROC SQL

Creating and Managing Tables Using PROC SQL

RangTech
Enhancing the way IT works

Method of Creating Table	Example
create an **empty table** by **defining columns**	proc sql; **create table** work.discount (Destination char(3), BeginDate num Format=date9., EndDate num format=date9., Discount num);
create an **empty table** that is **like** (has the same columns and attributes as) an existing table	proc sql; **create table** work.flightdelays2 **like** sasuser.flightdelays;
create a **populated table** (a table with both columns and rows of data) **from a query result**	proc sql; **create table** work.ticketagents **as** select lastname, firstname, jobcode, salary from sasuser.payrollmaster, sasuser.staffmaster where payrollmaster.empid = staffmaster.empid and jobcode contains 'TA';

Creating an Empty Table by Defining Columns

- You can create a table with no rows by using a CREATE TABLE statement that contains column specifications. A column specification includes the following elements: column name (required), data type (required), column width (optional), one or more column modifiers (optional), and a column constraint (optional).
- **Creating an Empty Table by Defining Columns**
- proc sql; create table work.discount (Destination char(3), BeginDate num Format=date9., EndDate num format=date9., Discount num); quit;

Creating an Empty Table That Is Like Another Table

- To create a table with no rows that has the same structure as an existing table, use a CREATE TABLE statement that contains the keyword LIKE. To specify a subset of columns to be copied from the existing table, use the SAS data set options DROP= or KEEP= in your CREATE TABLE statement.
- **Creating an Empty Table That is Like Another Table**
- proc sql; create table work.flightdelays2 (drop=delaycategory destinationtype) like sasuser.flightdelays; quit;

Creating a Table from a Query Result

To create a new table that contains both columns and rows that are derived from an existing table or set of tables, use a CREATE TABLE statement that includes the keyword AS and the clauses that are used in a query. This method enables you to copy an existing table quickly.

- EXAMPLE;
- proc sql; create table work.ticketagents as select lastname, firstname, jobcode, salary from sasuser.payrollmaster, sasuser.staffmaster where payrollmaster.empid = staffmaster.empid and jobcode contains 'TA'; quit;

Enhancing the way IT works

Displaying the Structure of a Table

- **To display, in the SAS log, a list of a table's columns and their attributes and other information about a table, use the DESCRIBE TABLE statement.**
- proc sql;
- describe table work.discount;
- quit;

PanoTech
Enhancing the way IT works

Inserting Rows of Data into a Table

The INSERT statement can be used in three ways to insert rows of data in existing tables, either empty or populated. You can insert rows by using

- the SET clause to specify column names and values in pairs
- the VALUES clause to specify a list of values
- the clauses that are used in a query to return rows from an existing table.

RangTech
Enhancing the way IT works

Inserting Rows of Data into a Table

- **Inserting Rows into a Table by Specifying Column Names and Values**
- proc sql; insert into work.discount set destination='LHR', begindate='01MAR2000'd, enddate='05MAR2000'd, discount=.33 set destination='CPH', begindate='03MAR2000'd, enddate='10MAR2000'd, discount=.15; quit;

Ranglech
Enhancing the way IT works

Inserting Rows of Data into a Table

- **Inserting Rows into a Table by Specifying Lists of Values**
- proc sql; insert into work.discount (destination, begindate,enddate, discount) values ('LHR','01MAR2000'd, '05MAR2000'd,.33) values ('CPH','03MAR2000'd, '10MAR2000'd,.15); quit;

RangTech
Enhancing the way IT works

Inserting Rows of Data into a Table

- **Inserting Rows into a Table from a Query Result**
- proc sql; insert into work.payrollchanges2 select empid, salary, dateofhire from sasuser.payrollmaster where empid in ('1919','1350','1401'); quit;

INTEGRITY CONSTRAINTS

- **Creating a Table That Has Integrity Constraints**

- Integrity constraints are rules that you can specify in order to restrict the data values that can be stored for a column in a table. SAS enforces integrity constraints when values associated with a column are added, updated, or deleted. Integrity constraints help you preserve the validity and consistency of your data.
- You can create integrity constraints by using either **PROC SQL** or **PROC DATASETS**. PROC DATASETS can assign constraints only to an existing table. PROC SQL can assign constraints either as it creates a new table or as it modifies an existing table. This lesson discusses the use of PROC SQL to create integrity constraints while creating a table.

Constraint Type	Action
CHECK	Ensures that a specific set or range of values are the only values in a column. It can also check the validity of a value in one column based on a value in another column within the same row.
NOT NULL	Guarantees that a column has non-missing values in each row.
UNIQUE	Enforces uniqueness for the values of a column.
PRIMARY KEY	Uniquely defines a row within a table, which can be a single column or a set of columns. A table can have only one PRIMARY KEY. PRIMARY KEY includes the attributes of the constraints NOT NULL and UNIQUE.
FOREIGN KEY	Links one or more rows in a table to a specific row in another table by matching a column or set of columns (a FOREIGN KEY) in one table with the PRIMARY KEY in another table. This parent/child relationship limits modifications made to both PRIMARY KEY and FOREIGN KEY constraints. The only acceptable values for a FOREIGN KEY are values of the PRIMARY KEY or missing values.

INTEGRITY CONSTRAINTS CONT.

- When you add an integrity constraint to a table that contains data, SAS checks all data values to determine whether they satisfy the constraint before the constraint is added.
  You can use integrity constraints in two ways, **general** and **referential**. General constraints enable you to restrict the data values accepted for a column in a single table. Referential constraints enable you to link the data values of a column in one table to the data values of columns in another table.

General Integrity Constraints

- **General Integrity Constraints**
- General integrity constraints enable you to restrict the values of columns within a single table. The following four integrity constraints can be used as general integrity constraints:
- CHECK
- NOT NULL
- UNIQUE
- PRIMARY KEY.
- A PRIMARY KEY constraint is a general integrity constraint if it does not have any FOREIGN KEY constraints referencing it. A PRIMARY KEY used as a general constraint, is a shortcut for assigning the constraints NOT NULL and UNIQUE.
-

RangTech
Enhancing the way IT works

- **Creating a Table That Has Integrity Constraints**
- proc sql; create table work.employees (Name char(10), Gender char(1), HDate date label='Hire Date' not null, constraint prim_key primary key(name), constraint gender check(gender in ('M' 'F'))); quit;

Displaying Integrity Constraints for a Table

- **Displaying Integrity Constraints for a Table**
- proc sql; describe table constraints work.discount4; quit;

RangTech
Enhancing the way IT works

Referential Integrity Constraints

- A referential integrity constraint is created when a PRIMARY KEY integrity constraint in one table is referenced by a FOREIGN KEY integrity constraint in another table. There are two steps that must be followed to create a referential integrity constraint:
- Define a PRIMARY KEY constraint on the first table.
- Define a FOREIGN KEY constraint on other tables.
-
- Integrity constraints
- follow ANSI standards
- cannot be defined for views
- cannot be defined for historical versions of generation data sets.

RangTech
Enhancing the way IT works

PROC SQL

Creating and Managing Indexes Using PROC SQL

RonTech
Enhancing the way IT works

Understanding Indexes

- An index is an auxiliary file that is defined on one or more of a table's columns, which are called key columns. The index stores the unique column values and a system of directions that enable access to rows in that table by index value.
- When an index is used to process a PROC SQL query, PROC SQL accesses rows directly (without having to read all the prior rows) instead of sequentially.
- You can create two types of indexes:

1. simple index (an index on one column)
2. composite index (an index on two or more columns).

- You can define either type of index as a unique index, which requires that values for the key column(s) be unique for each row.

RonaTech
Enhancing the way IT works

Simple and Composite Indexes

- A **simple index** is based on one column that you specify. The indexed column can be either character or numeric. When you create a simple index by using PROC SQL, you must specify the name of the indexed column as the name of the index.
- A **composite index** is based on two or more columns that you specify. The indexed columns can be character, numeric, or a combination of both. In the index, the values of the key columns are concatenated to form a single value.

RonTech
Enhancing the way IT works

Unique Indexes

- If you want to require that values for the key column(s) are unique for each row, you can create either a simple or a composite index as a **unique index**. Once a unique index is defined on one or more columns in a table, SAS will reject any change to the table that would cause more than one row to have the same value(s) for the specified column or composite group of columns.

RanaTech
Enhancing the way IT works

INDEX

- **Benefits of Using an Index** For PROC SQL, there are three main benefits to using an index to access data directly (instead of reading the data sequentially):
- A **small subset of data** (<15% of rows) can be accessed more quickly. (As the size of the subset increases, the advantage of using an index decreases.)

- **Equijoins** can be performed without internal sorts.

- **Uniqueness** can be enforced by creating a unique index.
- **Example: Using an Index to Access a Small Subset of Data**
- Suppose you are writing a query that references the table **Work.Payrollmaster**. (**Work.Payrollmaster** is a duplicate of the table **Sasuser.Payrollmaster**.) **Work.Payrollmaster** stores payroll information for employees, and it has a simple index defined on the column JobCode. Your query's WHERE clause expression references the key column:
  proc sql; select empid, jobcode, salary from work.payrollmaster **where jobcode='FA3'** order by empid;

RangTech
Enhancing the way IT works

Creating an Index

- To create an index on one or more columns of a table, use the CREATE INDEX statement.
  General form, CREATE INDEX statement: CREATE <UNIQUE> INDEX *index-name*
  ON *table-name* (*column-name-1*<**, ...***column-name-n*>); where
- **UNIQUE** is a keyword that specifies that all values of the column(s) specified in the statement must be unique.

- *index-name* specifies the name of the index to be created. If you are creating an index on one column only, then *index-name* must be the same as *column-name-1*. If you are creating an index on more than one column, then *index-name* cannot be the same as the name of any existing column or index in the table.

- *table-name* specifies the name of the table on which the index will be created.

- *column-name* specifies a column to be indexed. Columns can be specified in any order; however, column order is important for data retrieval. The first-named column is the primary key, the second-named column is the secondary key, and so on.

RangTech
Enhancing Skill Development

Example: Creating a Simple Index

- The following PROC SQL step uses the CREATE INDEX statement to create a simple, unique index that is based on the column EmpID in the temporary table **Work.Payrollmaster**. (**Work.Payrollmaster** is a duplicate of the table **Sasuser.Payrollmaster**.)
  proc sql; create unique index **EmpID** on work.payrollmaster(**empid**);The specified index name (**EmpID**) must be the same as the name of the key column.When this step is submitted, the SAS log displays the following message.

RanaTech
Enhancing the way IT works

Example: Creating a Composite, Unique Index

- The following PROC SQL step uses the CREATE INDEX statement to create the composite, unique index **daily** on the columns FlightNumber and Date:
  proc sql; create unique index **daily** on work.marchflights(**flightnumber,date**);When this step is submitted, the SAS log displays the following message.

- SAS Log
  NOTE: Composite index daily has been defined.

RangTech
Enhancing the way it works

Understanding the Costs of Using an Index

- When you are deciding whether to create an index, you should consider the associated increase in resource usage, which includes the following:
- **Additional CPU time** is necessary to create an index, to maintain the index when the table is modified, and to use an index to read a row from a table.

RangTech
Enhancing the way IT works

- Using an index to read rows from a table might require **additional I/O (input/output) requests** when compared to reading the table sequentially.

- Using an index requires **additional memory** for buffers into which the index pages and code are loaded for processing.

- **Additional disk space** is required to store the index file, which can show up as a separate file (in the Windows and UNIX operating environments, for example) or can appear to be part of the data file (in the z/OS (OS/390) operating environment).

Deciding Whether to Create an Index (continued)

- **Guidelines for Creating Indexes**
- To use indexes effectively, follow these guidelines for creating indexes:
- Keep the number of indexes to a minimum to reduce disk storage and update costs.

- Do not create an index for small tables. Sequential access is faster on small tables.

- Do not create an index based on columns that have a very small number of distinct values (low cardinality), for example, a Gender column that contains only the two values *Male* and *Female*.

- Use indexes for queries that retrieve a relatively small subset of rows—that is, less than 15%.

- Do not create more than one index that is based on the same column as the primary key.

RangTech
Enhancing the way IT works

Displaying Index Specifications

- To display a CREATE INDEX statement in the SAS log for each index that is defined for one or more specified tables, use the DESCRIBE TABLE statement.
- proc sql;
- describe table marchflights;
- quit;

PanGlech
Enhancing the way IT works

Dropping Indexes

To drop (delete) one or more indexes, use the DROP INDEX statement.

```
proc sql;
 drop index daily from work.marchflights;
quit;
```

RangTech
Enhancing the way Networks

Determining Whether SAS Is Using an Index

- By default, when a PROC SQL query or any other program is submitted in SAS, only notes, warnings, and error messages are written to the SAS log. To display additional messages, such as information about indexes that have been defined and that have been used in processing the program, specify the SAS system option **MSGLEVEL=I**. You specify the MSGLEVEL= option in the OPTIONS statement, before the PROC SQL statement

Controlling Index Usage

- To control index usage, use the **IDXWHERE=** and **IDXNAME=** SAS data set options to override the default settings. You can use either of these options, but you cannot use both options at the same time. As with other SAS data set options, you specify the IDXWHERE= or IDXNAME= option in parentheses after the table name in the FROM clause of a PROC SQL query.

Rangtech
Enhancing the way IT works

Using IDXWHERE=

- **Using IDXWHERE= to Direct SAS to Use or Not to Use an Index**
- The IDXWHERE= option enables you to override the decision that SAS makes about whether to use an index.
- **General form, IDXWHERE= option: IDXWHERE=**YES | NO;
- where
- YES tells SAS to choose the best index to optimize a WHERE expression, and to disregard the possibility that a sequential search of the table might be more resource-efficient.

RanaTech
Enhancing the way IT works

- NO tells SAS to ignore all indexes and satisfy the conditions of a WHERE expression with a sequential search of the table.
- proc sql; select * from marchflights **(idxwhere=no)** where flightnumber='182';

Using IDXNAME= to Direct SAS to Use a Specified Index

- The IDXNAME= option directs SAS to use an index that you specify, even if SAS would have selected **not** to use an index or to use a different index.
  General form, IDXNAME= option: IDXNAME=*index-name*; where *index-name* specifies the name of the index that should be used for processing.
- SAS uses the specified index if the following conditions are true:
- The specified index must exist.

RangTech
Enhancing the way IT works

- The specified index must be suitable by having at least its first or only column match a condition in the WHERE expression.
- proc sql; select * from marchflights **(idxname=daily)** where flightnumber='182';

Points to Remember

- **Points to Remember**
- An index cannot be created on a view.
- Keep the number of indexes to a minimum to reduce disk storage and update costs.
- Do not create an index for small tables; sequential access is faster on small tables.
- Do not create an index based on columns that have a very small number of distinct values (low cardinality), for example, a Gender column that contains only the two values *Male* and *Female*.
- Use indexes for queries that retrieve a relatively small subset of rows—that is, less than 15%.
- Do not create more than one index that is based on the same column as the primary key.

RangTech
Enhancing the way IT works

PROC SQL

MANAGING PROCESSING USING PROC SQL

RangTech
Enhancing the way IT works

Specifying SQL options

- The SQL procedure offers a variety of options that affect processing. Some options control execution. For example, you can limit the number of rows read or written during a query or limit the number of internal loops PROC SQL performs. Other options control output. For example, you can flow character columns, number your rows, or double-space output. Options are also available for testing and evaluating performance. Options are specified in the PROC SQL statement

RangTech
Enhancing the way IT works

Restricting Row Processing

- The OUTOBS= option restricts the number of rows that PROC SQL displays or writes to a table. The INOBS= option restricts the number of rows that PROC SQL takes as input from any single source. The INOBS= option is similar to the SAS system option OBS= and is useful for debugging queries on large tables.

Pangtech
Enhancing the way it works

Querying a Table Using PROC SQL Options

- proc sql outobs=5;
  select flightnumber, destination
  from sasuser.internationalflights;
  reset number;
  select flightnumber, destination
  from sasuser.internationalflights;
  where boarded gt 200; quit;

RangTech
Enhancing the way IT works

Controlling Output

- The NUMBER | NONUMBER option specifies whether the SELECT statement should include a column named ROW, which is the row number of the data as it is retrieved. NONUMBER is the default. The option is similar to the OBS | NOOBS option in the PRINT procedure.
- The DOUBLE | NODOUBLE option specifies whether PROC SQL output is double-spaced in the listing output. The default is NODOUBLE.
- The FLOW | NOFLOW | FLOW=*n* | FLOW=*n m* option controls the appearance of wide character columns in the listing output. The FLOW option causes text to be flowed in its column instead of wrapping the entire row. Specifying *n* sets the width of the flowed column. Specifying *n* and *m* floats the width of the column between limits to achieve a balanced layout.

Testing and Evaluating Performance

The STIMER | NOSTIMER option specifies whether PROC SQL writes timing information for each statement to the SAS log, in addition to writing a cumulative value for the entire procedure. NOSTIMER is the default. In order to use the STIMER option in PROC SQL, the **SAS system option** STIMER (the default) must also be in effect.

RanTech
Enhancing the way IT works

Resetting Options

After you specify an option, it remains in effect until you change it or you re-invoke PROC SQL. You can use the RESET statement to add, drop, or change PROC SQL options without re-invoking the SQL procedure.

Rangtech
Enhancing the way it works

Using Dictionary Tables

- SAS session metadata is stored in Dictionary tables, which are special, read-only SAS tables that contain information about SAS data libraries, SAS macros, and external files that are available in the current SAS session.

- A Dictionary table also contains the settings for SAS system options and SAS titles and footnotes that are currently in effect.

- Accessing a Dictionary table causes PROC SQL to determine the current state of the SAS session and return the information that you want. Dictionary tables can be accessed by running a PROC SQL query against the table, using the **Dictionary** libref. You can also access a Dictionary table by referring to the PROC SQL view of the table that is stored in the **Sashelp** library.

- To see how each Dictionary table is defined, submit a DESCRIBE TABLE statement. After you know how a table is defined, you can use its column names in a subsetting WHERE clause in order to retrieve specific information. To display information about the files in a specific library, specify the column names in a SELECT statement and the dictionary table name in the FROM clause. You can also use Dictionary tables to determine more specific information such as which tables in a SAS library contain a specific column. proc sql; describe table dictionary.columns; select memname from dictionary.columns where libname='SASUSER' and name='EmpID'; quit;

Additional Features

- The LOOPS= option restricts the number of iterations of the inner loop in PROC SQL. By setting a limit, you can prevent queries from consuming excessive resources.
- The ERRORSTOP | NOERRORSTOP option specifies whether PROC SQL stops executing if it encounters an error.

RangTech
Enhancing the way IT works

Points to Remember

After you specify an option, it remains in effect until you change it or you re-invoke PROC SQL.

- The DOUBLE | NODOUBLE and the FLOW | NOFLOW | FLOW=*n* | FLOW=*n m* options do not affect the appearance of HTML, PDF, or RTF output that is created with the Output Delivery System.

- If you query a Dictionary table about the files in a specific library, the library name used in the WHERE clause must be specified in uppercase letters because that is how it is stored in SAS. Column names used in the WHERE clause must be specified in the same case as they appear in the Dictionary table.

Ranglech
Enhancing the way IT works